

Tim Farnung, Dieter Grünke, Erik Sauer, Leonhard Schneider

Projekt: StudyConnect

Lab 6

Vulnerability Analysis: Risk R-02 (IDOR on PUT /api/tasks/<task_id>)

This report focuses on Risk R-02 identified in Lab 5: an Insecure Direct Object Reference (IDOR) in the task update flow (PUT /api/tasks/<task_id>) that allows unauthorized mutation of other users' tasks. The analysis below lists three distinct vulnerabilities / antipatterns that contribute to R-02 and describes exploitation paths and fix rationale for each. A refactored, secure version of `update_task_service` follows the analysis.

1) Missing Object-Level Authorization (IDOR)

- **Vulnerability class:** Broken Access Control / IDOR
- **How it can be exploited:** The API accepts a `task_id` path parameter and applies updates after basic validation, but does not reliably verify that the authenticated editor (the `editor_user_id`) is the owner of the target `task` or an authorized admin. An attacker who is authenticated as any user can iterate or guess `task_id` values and submit `PUT` requests to alter or delete other users' tasks.
- **Fix rationale:** Enforce object-level authorization before any mutation. The service must retrieve the task and immediately verify that `editor_user_id` equals `task.user_id` or that the editor holds explicit administrative privileges for the task's group. If the check fails, the service must abort with a 403/PermissionError without modifying any fields. This prevents IDOR by design and centralizes the ownership check at the service boundary.

2) Mass Assignment of Protected Fields (user_id / group_id)

- **Vulnerability class:** Mass Assignment / Insecure Deserialization
- **How it can be exploited:** The current update path applies client-supplied fields to model attributes (e.g., `user_id`, `group_id`) via dynamic assignment. An attacker can include `user_id` in the JSON payload to change the owner of a task or include `group_id` to move a task into another group's context, gaining access or control they should not have.
- **Fix rationale:** Implement strict field whitelisting and ignore protected fields from client payloads. Accept only explicit editable attributes (title, kind, status, priority, notes, assignee, progress, deadline). Any server-controlled ownership fields such as `user_id` or `group_id` must be set only via dedicated, permission-checked server-side flows. This eliminates mass-assignment as an attack surface.

3) Trusting Client-Controlled Ownership / Incomplete Membership Checks

- **Vulnerability class:** Authorization Logic Flaw / Insecure Trust
- **How it can be exploited:** Even if `group_id` changes are validated in part, the code may rely on weak membership checks or accept group assignment without validating the editor's membership/role for the target group. An attacker can craft requests that appear to satisfy superficial checks and assign tasks to groups or assignees that they shouldn't control.

- **Fix rationale:** Require explicit proofs of authorization for any operation that affects ownership or group association. For group assignment, require that the editor is a member of the target group (and optionally has admin privileges) before allowing the change. For assignee changes, ensure the assignee is a valid member of the target group. Keep ownership decisions out of the client's control and centralize all membership checks in the service layer.

Refactored Code diff: secure `update_task_service`

The diff below is a refactored, self-contained implementation for `update_task_service` that demonstrates the recommended mitigations:

```
def update_task_service(task_id, data, editor_user_id=None):
    task = db.session.get(Task, task_id)
    if not task:
        raise Exception(f"Task with id {task_id} does not exist")

    - # Pre-cleanup: The 'expired' status is a pure display state of the
    frontend
    - # and must never be written to or validated by the database.
    + # Require authenticated editor for any state-changing operation
    + if not editor_user_id:
    +     raise PermissionError("Authentication required to modify tasks")
    +
    + editor = db.session.get(User, editor_user_id)
    + if not editor:
    +     raise PermissionError("Editor user not found")
    +
    + # Object-level authorization: editor must be the owner or a group
    admin
    + is_owner = (str(task.user_id) == str(editor_user_id))
    + is_group_admin = any(getattr(m, 'role', None) == 'admin' and
    getattr(m, 'group_id', None) == task.group_id for m in getattr(editor,
    'group_memberships', []))
    + if not (is_owner or is_group_admin):
    +     raise PermissionError("You are not allowed to modify this task")
    +
    + # Field whitelist: only allow specific editable fields from client
    payload
    + allowed_fields = {'title', 'kind', 'priority', 'status', 'assignee',
    'notes', 'progress', 'deadline'}
    + sanitized = {k: v for k, v in data.items() if k in allowed_fields and
    v is not None}

    + # Prevent frontend-only display states from being written
    + if sanitized.get('status') == 'expired':
    +     sanitized.pop('status', None)
    - if data.get('status') == 'expired':
    -     del data['status']

    # Validate status transition
    + if 'status' in sanitized:
```

```

-     if 'status' in data:
-         # Normalize both the current and the new status to resolve
inconsistencies
        current_status = task.status.lower().replace("inprogress",
"in_progress").replace("expired", "todo")
+     new_status = sanitized['status'].lower().replace("inprogress",
"in_progress")
-         # Convert the received status to lowercase to avoid case issues
(e.g., inProgress vs. in_progress)
-         new_status = data['status'].lower().replace("inprogress",
"in_progress")
-         # Save the corrected version back to the data so it is stored
correctly
+         sanitized['status'] = new_status
-         data['status'] = new_status
-
-         # Only validate the transition if the status actually changes.
        if new_status != current_status and new_status not in
VALID_STATUSES.get(current_status, []):
            raise ValueError(f"Invalid status transition from {current_status}
to {new_status}")

-         # Prevent a past-due task from being started
        if new_status == 'in_progress' and task.deadline < date.today():
            raise ValueError("Cannot start a task that is past its deadline.")

-         # If a task is set to 'done', automatically set its progress to
100.
        if new_status == 'done':
            task.progress = 100

        # Validate progress
-         if 'progress' in data:
-             progress = data['progress']
+         if 'progress' in sanitized:
+             progress = sanitized['progress']
            if not (0 <= progress <= 100):
                raise ValueError("Progress must be between 0 and 100")

        # Validate priority
+         if 'priority' in sanitized and sanitized['priority'] not in
VALID_PRIORITIES:
-             if 'priority' in data:
-                 if data['priority'] not in VALID_PRIORITIES:
-                     raise ValueError(f"Invalid priority value. Must be one of:
{VALID_PRIORITIES}")
+                 raise ValueError(f"Invalid priority value. Must be one of:
{VALID_PRIORITIES}")

+         # Validate assignee membership server-side (if provided)
-         # Validate group assignment permission
-         if data.get('group_id') is not None:
-             if not editor_user_id:
-                 raise PermissionError("User ID is required to assign a task to

```

```

a group.")
-
-     editor = db.session.get(User, editor_user_id)
-     if not any(m.group_id == data['group_id'] for m in
editor.group_memberships):
-         raise PermissionError("You can only assign tasks to groups you
are a member of.")
-
-     # Validate assignee
-     if data.get('assignee') is not None:
-         assignee = db.session.get(User, data['assignee'])
+     if 'assignee' in sanitized and sanitized['assignee'] is not None:
+         assignee = db.session.get(User, sanitized['assignee'])
+         if not assignee:
+             raise ValueError("Assignee user not found")

-     # The group_id to check against is either the new one from `data`
or the existing one on the task
-     target_group_id = data.get('group_id', task.group_id)
-     if target_group_id and not any(m.group_id == target_group_id for m
in assignee.group_memberships):
-         raise ValueError("Assignee must be member of the group")
+     target_group_id = task.group_id
+     if target_group_id and not any(m.group_id == target_group_id for m
in getattr(assignee, 'group_memberships', [])):
+         raise ValueError("Assignee must be member of the task's
group")

+     # Apply sanitized updates only
+     for field, value in sanitized.items():
+         if field == 'deadline':
-         # Update fields
-         for field in ['title', 'kind', 'priority', 'status', 'user_id',
'group_id', 'assignee', 'notes', 'progress']:
-             if field in data and data[field] is not None:
-                 setattr(task, field, data[field])
-
-         if 'group_id' in data: # Explicitly handle group_id to allow 'None'
-             task.group_id = data['group_id']
-
-         if 'deadline' in data:
-             deadline_date = datetime.strptime(data['deadline'], '%Y-%m-
%d').date()
-             if deadline_date < date.today():
-                 raise ValueError("Deadline cannot be in the past")
-             task.deadline = deadline_date
+             deadline_date = datetime.strptime(value, '%Y-%m-%d').date()
+             if deadline_date < date.today():
+                 raise ValueError("Deadline cannot be in the past")
+             task.deadline = deadline_date
+         else:
+             setattr(task, field, value)

```

```
db.session.commit()  
return task
```